

From Imperative to Declarative: Towards LLM-friendly OS Interfaces for Boosted Computer-Use Agents

Yuan Wang^{★♦†} Mingyu Li^{★♦} Haibo Chen^{★♦‡}

[★]Key Laboratory of System Software (Chinese Academy of Sciences)

[♦]Institute of Software Chinese Academy of Sciences

[‡]Shanghai Jiao Tong University

[†]University of Chinese Academy of Sciences

Abstract

Computer-use agents (CUAs) powered by large language models (LLMs) have emerged as a promising approach to automating computer tasks, yet they struggle with the existing human-oriented OS interfaces—graphical user interfaces (GUIs). GUIs force LLMs to decompose high-level goals into lengthy, error-prone sequences of fine-grained actions, resulting in low success rates and an excessive number of LLM calls.

We propose Declarative Model Interface (DMI), an abstraction that transforms existing GUIs into three declarative primitives: access, state, and observation, thereby providing novel OS interfaces tailored for LLM agents. Our key idea is policy-mechanism separation: LLMs focus on high-level semantic planning (policy) while DMI handles low-level navigation and interaction (mechanism). DMI does not require modifying the application source code or relying on application programming interfaces (APIs).

We evaluate DMI with Microsoft Office Suite (Word, PowerPoint, Excel) on Windows. Integrating DMI into a leading GUI-based agent baseline improves task success rates by 67% and reduces interaction steps by 43.5%. Notably, DMI completes over 61% of successful tasks with a single LLM call.

CCS Concepts: • Software and its engineering → Operating systems; • Computing methodologies → Machine learning approaches; Intelligent agents.

Keywords: OS abstraction, Interface design, LLM-friendly OS interfaces

ACM Reference Format:

Yuan Wang^{★♦†} Mingyu Li^{★♦} Haibo Chen^{★♦‡}. 2026. From Imperative to Declarative: Towards LLM-friendly OS Interfaces for Boosted Computer-Use Agents. In *21st European Conference on Computer Systems (EUROSYS '26)*, April 27–30, 2026, Edinburgh, Scotland



This work is licensed under a Creative Commons Attribution 4.0 International License.

EUROSYS '26, Edinburgh, Scotland UK

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2212-7/2026/04

<https://doi.org/10.1145/3767295.3803576>

Uk. ACM, New York, NY, USA, 15 pages. <https://doi.org/10.1145/3767295.3803576>

1 Introduction

Computer-use agents (CUAs) powered by large language models (LLMs) demonstrate remarkable potential in automating complex workflows, creating unprecedented opportunities for productivity enhancement. Recent breakthroughs have catalyzed intense interest across both industry and academia [1, 11, 13, 18, 25, 26, 32, 37, 44, 48, 51]. To interact with applications running on computer systems, state-of-the-art CUAs primarily rely on two interfaces: application programming interfaces (APIs) and graphical user interfaces (GUIs).

Under the first paradigm, API-based approaches capitalize on the code-generation capabilities of LLMs to directly invoke application-specific APIs [19, 20, 33, 51]. While this approach typically yields higher success rates and requires fewer execution steps, it faces a critical limitation: many applications lack publicly available APIs, thereby restricting the approach's overall generality. On the other hand, GUI-based approaches [26, 32, 37, 51, 52] leverage LLMs (including multimodal LLMs) to perceive screen content, locate interface elements, and emulate human actions such as clicks and scrolls. This approach offers unparalleled generality since GUIs represent the dominant human-computer interface paradigm on modern desktop and mobile operating systems. However, it demands that LLMs generate lengthy, fine-grained action sequences to manipulate UI control elements (hereafter, controls), increasing LLM inference costs while compounding the risk of cascading failures [14, 20, 40, 47, 49].

Our investigation suggests that the need for complex action sequences in GUI use stems from the **imperative** nature of GUI design. First, unlike APIs with direct function calls, GUIs require navigating through spatial layouts to reveal hidden or off-screen controls. This forces the LLM to execute chains of **navigation** actions (e.g., navigating menus, tabs, and dropdowns) to make the control visible. Second, GUI controls require explicit **interaction** (e.g., a click) to trigger a function. Beyond simple clicks, many controls demand composite interactions. For instance, manipulating a scrollbar requires a coordinated sequence of actions: moving the cursor to the scrollbar, pressing the mouse button, dragging

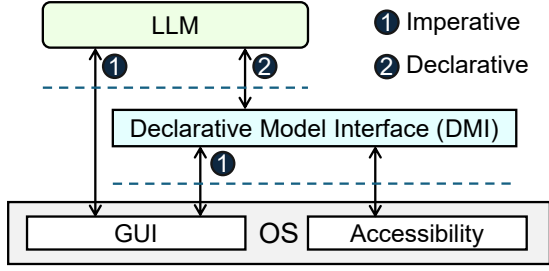


Figure 1. Overview of the DMI abstraction layer. DMI is based on ubiquitous GUI and OS accessibility features [2, 5, 6]. *Declarative* specifies the intended state or outcome; *imperative* enumerates the actions that realize it. DMI abstraction provides novel **OS interfaces** tailored for LLM agents.

to a target position while holding the button, and finally releasing it.

This imperative paradigm creates an entanglement of two distinct aspects in GUI application use. The first is the orchestration of application functionality according to task semantics, which we term **policy**. The second is the process of using functional controls, which involves navigation and interaction, an aspect we refer to as **mechanism**.

In current GUIs, policy is tightly coupled with mechanism: users cannot directly invoke the function; they must perform the requisite navigation and interaction. This coupling poses a significant bottleneck for LLMs; it substantially increases task complexity, strains the LLM’s spatial reasoning and visual processing, and necessitates a higher number of LLM invocations, inflating both token costs and latency.

This raises an essential research question: *How should OSes evolve interfaces to serve a fundamentally new class of OS users (LLMs), instead of forcing them to mimic humans and use existing human-oriented interfaces (namely, GUIs) for which they are ill-suited?*

Our key insight is that *while LLMs struggle with fine-grained mechanisms, a substantial fraction of the interaction logic with GUI is deterministic and can be executed independently of the LLM*. Based on this observation, we introduce Declarative Model Interface (DMI), an abstraction layer that transforms complex GUI navigation and interaction into three declarative primitives: **access**, **state**, and **observation**. By handling navigation and interaction deterministically, DMI decouples policy from mechanism—a separation of concerns that shifts the LLM’s role from “orchestrate both high-level functions and low-level UI actions” to “orchestrate only the semantic, non-deterministic aspects of the task.”

Unlike imperative GUI interaction, declarative DMI allows and requires the LLM to specify the desired outcome directly, rather than emitting concrete actions to realize the outcome. Using DMI’s declarative primitives, the LLM can express the intended result for a control: the control is to be “accessed” or “set to a target state”, or to have its information “observed

and retrieved”. DMI then executes the necessary steps to achieve that result. This procedure is fully transparent to the LLM. Such a design meets our goal: to let the LLM abstract away from concrete GUI operations, and eventually focus on semantic planning for the user’s task.

There are several challenges to be addressed to realize DMI. First, the controls’ navigation relationships are typically implicit within the applications, which obstructs navigation abstraction. Second, a control’s functionality is path-dependent. Take Microsoft Word as an example: accessing a standard color control via the Font, Outline, or Underline path can yield different semantics. Enforcing path uniqueness through duplication (i.e., cloning the merge node and its substructure) can cause an exponential blow-up in an already large application node set, which in turn highlights another major issue: LLM context is limited and costly. DMI design must account for invocation cost. Additionally, the ideal interface should provide a stable abstraction: this requires DMI to handle the non-determinism of GUI execution robustly. Finally, an LLM-friendly DMI must also consider the caller’s (the LLM’s) imperfect instruction-following. For example, while we explicitly instruct the LLM to output target function controls only, the LLM may still output navigation paths.

We present an end-to-end solution to these challenges. Specifically, we model controls’ navigation relationships and transform the resulting graph into a path-unambiguous topology (§ 3.2). We introduce an LLM-context-friendly representation of controls and navigation, and design an on-demand querying mechanism to balance context cost, function coverage, and efficiency (§ 3.3). DMI is designed with an “LLM-as-user” assumption; to mitigate imperfect instruction-following, we apply filtering to ensure DMI entirely takes over the navigation process. To provide a stable declarative abstraction, DMI incorporates robustness mechanisms to handle instability in executing UI action sequences (§ 3.4). We enforce a strict separation between control access and complex interactions to relieve the tension between accuracy and efficiency (§ 3.4, § 3.5).

We validate DMI through a set of case studies against Microsoft Word, Excel, and PowerPoint. These applications cover a diverse range of real-world scenarios, including text editing, spreadsheet manipulation, and graphics processing. Our evaluation with the OSWorld-W [44] dataset shows that DMI outperforms UFO-2 [51], the leading GUI-based baseline, by increasing the absolute task success rate by 29.6% (67% relative improvement), reducing interaction steps by 43.5%, and decreasing completion time by 39%. Regarding failure cases, the UFO-2 baseline exhibits numerous failures related to the mechanism, such as errors in visual recognition, fine-grained interaction, and navigation. In contrast, with DMI, over 80.9% of failures are policy-related (e.g., LLM semantic misunderstandings), rather than mechanism-related. These results demonstrate the effectiveness of declarative interfaces as LLM-friendly interaction paradigms.

In this paper, we make the following contributions:

- We identify that human-oriented OS interfaces (i.e., GUIs) pose challenges for LLM-driven CUAs, a fundamentally new class of OS users with distinct characteristics and constraints.
- We introduce DMI, an abstraction that decouples policy from mechanism by reducing complex GUI operations to declarative primitives, enabling LLMs to focus on semantic planning. DMI provides LLM-friendly OS interfaces.
- We conduct an extensive evaluation showing that DMI significantly improves LLM task success rates and efficiency compared to state-of-the-art GUI-based approaches.

2 From Imperative to Declarative

2.1 Human-centric Design: A Hurdle for LLMs

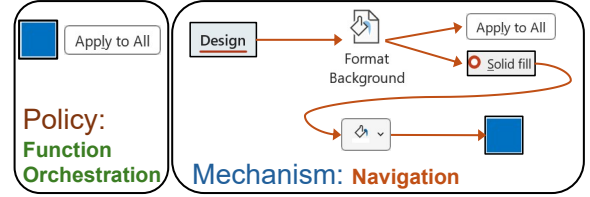
We first examine the paradox of GUIs: why the human-centric OS interface design, which is intuitive for humans, creates significant obstacles for LLMs.

Mismatch #1: Procedural navigation. GUI applications expose functionality in a step-by-step manner: controls are hierarchically organized behind menus, tabs, and dialog boxes. This design deliberately narrows the set of available choices at each step, decomposing control localization into low-cognitive-load decisions [30], while favoring visual recognition over memory-based recall [34]. The design assumes users (i) struggle with large decision spaces due to limited working-memory capacity, (ii) have poor exact syntax recall (e.g., command names, argument order), but (iii) excel at visual recognition. These assumptions do not hold for LLMs. LLMs can leverage a much larger context window (e.g., GPT-5, 400K tokens) and reliably produce grammar-constrained, structured outputs [27, 38], but they have comparatively weaker visual perception [15, 23, 50]. Given a global view, whether from trained knowledge or task-specific documentation in the prompt, an LLM can directly identify the appropriate control and generate structured invocations [28]. However, an imperative GUI requires LLMs to first produce navigation sequences that make the target controls visible. This increases the length of action chains and introduces systemic fragility, where any planning or execution error can cascade into complete task failure.

Mismatch #2: Iterative interaction. Many GUI controls rely on iterative interaction. For example, text selection involves repeated cursor movements to establish start and end positions. This design enforces high-frequency “observe-act” loops that transform challenging, high-precision outputs (exact coordinates) into manageable visual judgments (“is the current state acceptable?”). This assumes (i) frequent “observe-act” loops incur minimal cost, and (ii) perception is effortless and accurate. Neither assumption holds for LLMs. LLM inference latency makes frequent closed-loop control prohibitively expensive: while humans perform 3–5 mouse

Table 1. Task examples of imperative GUI vs. declarative DMI.

Task	GUI	DMI
1	click("Design") → click("Format Background") → click("Solid fill") → click("Fill Color") → click("Blue") → click("Apply to All")	visit(["Blue", "Apply to All"])
2	iterative interaction (drag and drop)	set_scrollbar_pos(80%)



(a) Task 1: make the background blue on all slides.



(b) Task 2: show the area close to the end.

Figure 2. Policy-Mechanism coupling in GUI use.

adjustments per second with real-time feedback, LLM inference requires 10–120+ seconds per round-trip [8, 10]. In addition, LLMs exhibit limited visual acuity, making precise UI understanding unreliable [15, 23, 50].

2.2 Insights

Insight #1: Policy-mechanism decoupling. When using GUI applications, **policy** (function orchestration) becomes tightly coupled with the **mechanism** (control navigation and interaction). Figure 2 and Table 1 illustrate this coupling through two real-world examples. Since human-centered GUIs misalign with LLM capabilities, LLMs perform poorly on mechanism-level tasks. In Task 1, invoking “Blue” requires emitting a precise five-step navigation sequence; any single misstep could invalidate the plan. In Task 2, the LLM must perform the iterative drag-observe cycle on the scrollbar and rely on visual feedback, while the LLM is prone to misperceiving on-screen positions. Additionally, many LLM calls increase latency. However, much of this mechanism is **deterministic** and can be resolved algorithmically without LLM involvement. Rather than forcing LLMs to plan fragile navigation sequences and complex interactions, we propose offloading deterministically solvable mechanisms to an abstraction layer, thus enabling LLMs to focus on non-deterministic, policy-level decisions that require semantic reasoning.

Insight #2: Deterministic navigation. Given a target control, computing an access path and navigating to it becomes a deterministic problem. Once an application is released, its control transitions form a finite state machine [42], with

control reachability and dependencies modeled as a directed graph where nodes represent controls. While controls may be accessible through multiple paths, removing back-edges and duplicate nodes with their successors yields a tree structure, producing a **unique** path identifiable from the control's identifier alone. This approach eliminates navigation dependencies from GUI use and compresses lengthy action chains. The LLM needs only to declare the target control rather than specify the concrete navigation sequence, removing responsibility for path correctness from the model.

Insight #3: Finite interaction operations. While controls exhibit diverse behaviors, Windows UI Automation (UIA) categorizes them into finite sets: 41 control types (e.g., Button, ListItem, Edit) and 34 control patterns (e.g., TextPattern, ScrollPattern, SelectionPattern) [29]. These universals enable us to abstract low-level, fine-grained interactions into state and result declarations such as `set_scrollbar_pos(x_percent, y_percent)` or `select_lines(start_index, end_index)`. With these wrapped primitives, LLMs only need to specify the desired control state, eliminating requirements for precise visual-coordinate reasoning or high-frequency interaction.

2.3 LLM-friendly Paradigm: Declarative Interfaces

Decoupling **policy** (what to do) from **mechanism** (how to do it) allows for a shift from an **imperative** to a **declarative** interface paradigm. In this paradigm, an interface user (in this case, an LLM) can specify a desired state rather than planning and executing a long sequence of imperative actions. For example, the LLM could simply declare its desired outcome, such as “set the *scrollbar* position to 50%”, or “access the *Apply to All* control”. The interface then handles the complex, low-level execution, regardless of the control's current state or visibility on the screen. Figure 2 illustrates this paradigm shift.

Declarative interfaces are well-suited for LLMs for two main reasons. First, this approach shifts interaction from state-based “observe-act” loops to target state setting. This is crucial because it eliminates the dependency on the LLM's relatively weaker precise visual perception and minimizes the need for high-frequency, real-time operations. Instead of constantly analyzing the screen to decide the next step, the LLM simply states the end goal. Second, by hiding the low-level, fine-grained details of interaction, this paradigm distills tasks down to their semantic reasoning, which enables LLMs to focus on their strengths: understanding high-level intent and emitting well-formed, grammar-constrained outputs.

2.4 Challenges

To realize the shift from imperative to declarative interaction, we need to address the following challenges:

Challenge #1: Navigation path ambiguity. Declarative control access requires explicit application navigation pathways, but these relationships exist implicitly and are not exposed as explicit data structures. Even with deterministic UI topology, *path ambiguity* exists: cycles and merge nodes in the navigation graph prevent mapping controls to unique access paths.

Cycles: Loops (e.g., $A \rightarrow B \rightarrow A$) may yield infinite traversal sequences.

Merge nodes: Controls may be reachable via multiple paths (e.g., $A \rightarrow C$ and $B \rightarrow C$).

Identical controls can enact different functions depending on path-related context. For instance, in Word's color picker, a color cell is reachable via “Font Color,” “Outline Color,” or “Underline Color”; the chosen path determines which property is modified.

Challenge #2: Limited LLM context windows. Navigation topology should be converted to textual prompts for LLM comprehension. However, modern applications expose numerous controls (>5,000 in Microsoft Office), making full topology prohibitively expensive for LLMs. An ideal interface should enable LLMs to accurately identify required controls without overwhelming their context windows.

Challenge #3: Inaccurate long-horizon planning. Since the navigation topology is deterministic, the LLM can plan multiple commands in a single call, even when target controls are not currently visible. However, this makes subsequent commands contingent on earlier correctness: unexpected intermediate outcomes can invalidate subsequent steps and cascade into task errors.

Additionally, real-world UI interaction is inherently unstable; for example, control name variations can break element identification.

3 The DMI Design

3.1 Overview

The GUI mechanism includes control navigation and interaction. DMI abstracts navigation as *access* declaration, and abstracts interaction as *state* and *observation* declarations.

- *Access declaration:* Given a control identifier, DMI deterministically navigates from any current state to that control and performs a primitive interaction (e.g., click).
- *State declaration:* Given a desired control end state (e.g., scrollbar position; selection state for a control or for text), DMI transitions the control from any current state to the target state, abstracting away compound interactions such as drag and keyboard–mouse coordination.
- *Observation declaration:* Given an information request (e.g., a control's text content), DMI returns structured data rather than relying on pixel-level recognition, without requiring compound interactions to reveal hidden content (e.g., expanding a table item).

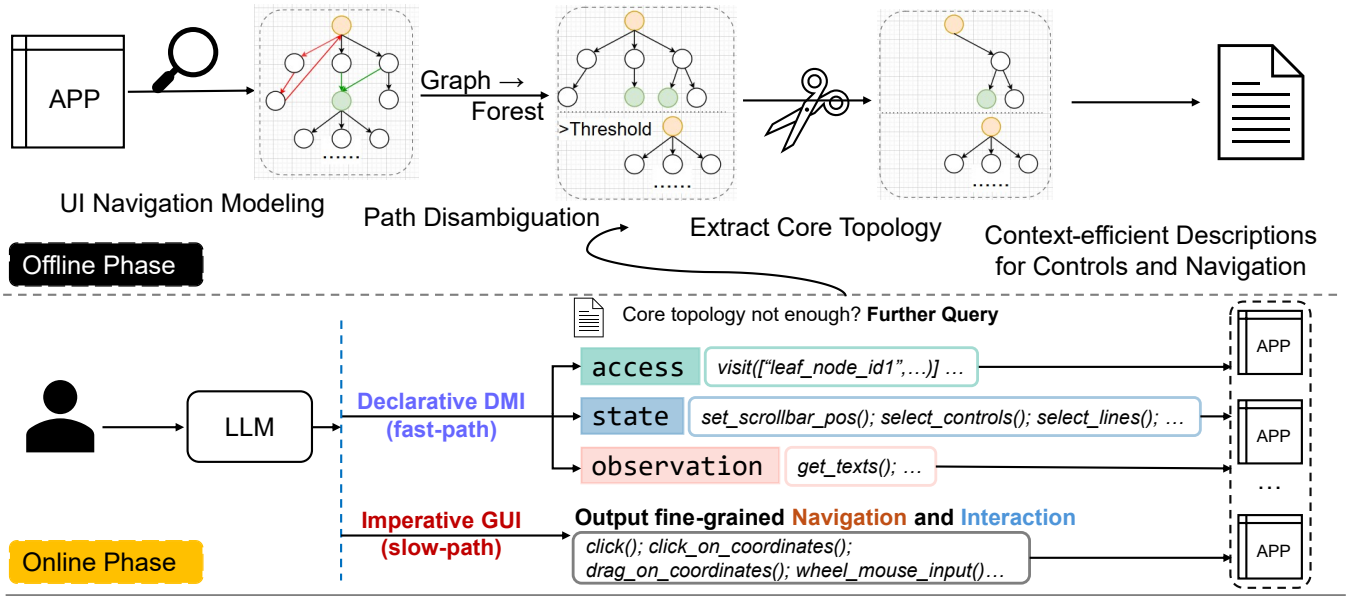


Figure 3. DMI Workflow: Offline Modeling and Online Execution. DMI has three declarative primitives: Access, State, and Observation.

Interfaces: DMI provides two categories of interfaces. The visit interface is designed to take over navigation and realize *access* declaration. Additionally, DMI presents a set of interaction-related interfaces to support *state* and *observation* declaration. These interfaces simplify complex interactions such as scrolling, selection, and text manipulation into direct state setting and structured information retrieval.

Workflow: The workflow of DMI is shown in Figure 3. First, navigation relationships among controls are modeled as a graph, which is further translated into a path-unambiguous forest containing a main tree and shared subtrees. Next, this forest topology is converted into context-efficient textual representations with query-on-demand mechanisms to reduce token overhead. Finally, LLM is provided with the declarative interface to use intended controls.

Outline: § 3.2 introduces the navigation topology addressing path ambiguity (Challenge #1). § 3.3 presents context-efficient descriptions for managing the LLM context window (Challenge #2). § 3.4 and § 3.5 detail interface design for robust and efficient execution (Challenges #3).

3.2 Path-Unambiguous Navigation Topology

This section presents the application navigation modeling and how to construct the path-unambiguous topology.

Navigation modeling: Inspired by prior work [22, 24], we build an automated prototype to model UI navigation relationships, with modest human intervention (detailed in § 4.1). The result is a UI Navigation Graph (UNG): nodes are UI controls and directed edges denote “click” interaction.

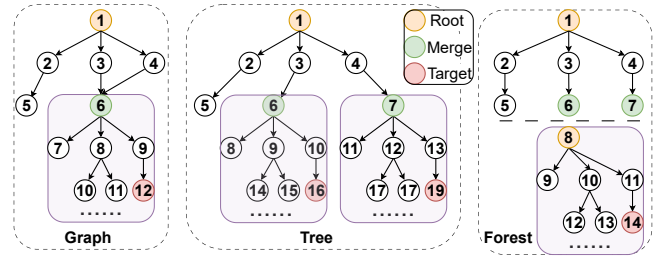


Figure 4. Navigation Topology. To access the bottom-right node along node 4, imperative GUI navigation relies on graph and requires the explicit path $\langle 1, 4, 6, 9, 12 \rangle$. With declaration, only $\langle 19 \rangle$ is required by tree, while the number of nodes explodes. For forest, the path required is $\langle 7, 14 \rangle$.

UNG is a directed graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ where each node $v \in \mathcal{V}$ corresponds to a UI control exposed by the accessibility API, and each edge captures **click**-induced reachability between controls. In UNG, we only model control-to-control transition relationships; we do not include keyboard-shortcut actions as transition edges, since their effects are typically achievable via equivalent click operations.

From directed graph to forest: To ensure unambiguous control access by declaring the control ID, UNG must be transformed into an *ambiguity-free* topology. We define path ambiguity-free as follows: for any given control, a unique access path can be determined within its connected topology. The resulting structure is a forest, comprising a main tree and a set of shared subtrees. This transformation involves two critical steps. First, we decycle the graph to a DAG. This process begins by removing cycles from the single-source

UNG to produce a single-source DAG. This is achieved by identifying and removing back-edges that form the cycles.

The next step is to solve path disambiguation by turning the DAG into a forest, which focuses on handling merge nodes in the DAG (nodes with multiple incoming edges). A naive approach would be to convert a merge node into a single tree by cloning the node and its entire descendant substructure for every incoming edge. While this guarantees unique paths, it causes an exponential node blow-up. For complex applications (e.g., Microsoft Office), the resulting description significantly increases the context size, which exceeds the available context window of the LLM (400K tokens, GPT-5) in our experiment. As discussed, simply deleting in-edges to enforce uniqueness is not an option, because different paths to the same GUI control can carry distinct semantics.

We designed a cost-based selective externalization algorithm to balance output-path length and total node count by transforming the single-source DAG into a forest. This bottom-up algorithm processes nodes in reverse topological order. For each merge node, it estimates the rooted substructure size and cloning cost—additional nodes from duplicating the substructure along all incoming edges. When this cost exceeds a configurable threshold, the node and descendants are externalized as a shared subtree, with incoming edges redirected to new reference nodes for indirect access. Otherwise, the substructure is cloned along each edge. This approach ensures linear node growth. In practice, the LLM specifies only a target control ID (see Figure 4) and reference IDs (typically one) for controls in shared subtrees. The executor then deterministically resolves navigation with reduced context overhead and bounded topology size while preserving semantic correctness and avoiding path ambiguity.

3.3 Context-Efficient Descriptions for Controls and Navigation

This section explains how DMI converts the navigation topology into a textual format optimized for LLMs.

Strawman: A straightforward approach would discard navigation controls and abstract functional controls (the leaf nodes of the topology) into a flattened, API-like list of endpoints. However, this approach sacrifices critical semantic information. Many UI controls share generic names (e.g., “Color,” “Settings,” or “OK” buttons), and a flattened list introduces ambiguity that increases the risk of incorrect LLM selections. To resolve this ambiguity within a flat structure, one would need to encode the navigation path or its semantic meaning into each leaf node’s description. This leads to massive data redundancy, as sibling nodes would share identical ancestor path information.

Compressed, hierarchical description: We encode the navigation hierarchy as compact, structured text. We retain the non-leaf nodes (navigation controls) as an integral part

of each control’s description. A control is thus described by a combination of its own properties and its hierarchical path. This keeps the organization of functionality explicit, allowing the LLM to disambiguate by full path, perform precise semantic reasoning, and avoid the duplication that a flattened representation would impose. DMI relies on a shared subtree entry map to connect the main tree and shared subtrees. This map records each reference node and the root of the subtree it targets. With this map and topology, the LLM can use `ref_id` when locating controls within shared subtrees, ensuring a unique and correct navigation.

Query on demand: To conserve costly LLM context windows, we adopt a layered retrieval strategy based on the observation that most tasks require only partial navigation topology. First, DMI by default provides a limited-depth core (e.g., six levels) instead of the complete forest, excluding large enumerations (font lists) and manually identified nodes. While pruning rules are currently manual, future versions could leverage LLM assistance for automation. Second, when the pruned core lacks required structure, the LLM requests additional content via `further_query` commands supporting two modes: (a) targeted branch queries expanding substructures beneath specified nodes, and (b) global queries retrieving the complete forest. This design minimizes initial context load for routine tasks while enabling access to additional structures through targeted expansion or complete forest retrieval when necessary.

3.4 Access Declaration: The visit Interface

For *access* declaration, the *visit* interface enables callers to directly access target controls via identifiers rather than navigation sequences. The *visit* interface also encompasses fundamental interactions such as clicks to improve efficiency and minimize cascading *visit* command calls.

Interface specification: *visit* accepts an array of structured commands and executes them sequentially in a single call. Specifically, it supports four categories of commands:

- **Control access:** Navigate to the target control and perform a primitive interaction (i.e., click).

Target in the main tree:

```
{"id": "<target_id>"}
```

Target in a shared subtree:

```
{"id": "<target_id>",  
  "entry_ref_id": ["<ref_id>", "..."]}
```

- **Access-and-input text:** Access an Edit type control and input text:

```
{"id": "<target_id>", "text": "<text>"}
```

- **Shortcut keys (auxiliary):** Issue a keyboard shortcut when a GUI click is insufficient (e.g., pressing ENTER to commit an edit):

```
{"shortcut_key": "<key_combination>"}
```

- **FurtherQuery:** Request additional topology when the default core topology is insufficient. This command is exclusive and cannot be mixed with other commands in the same call. “-1” refers to fetching the entire forest.

```
{"further_query": ["<node_id>", "..."]}
```

Balancing efficiency and accuracy: We address the efficiency-accuracy tension in *visit* through three approaches. First, *visit* bundles “navigation” and “primitive interaction” (e.g., click) into the *atomic* control-access command and supports multiple commands per call (e.g., *visit*([command1, command2])), enabling the LLM to execute several functions in one turn. Second, *visit* excludes complex interactions (i.e., actions supported by state declaration). When subsequent steps require composite interactions, DMI requires the LLM to stop emitting further *visit* commands. This forces the LLM to observe (e.g., verify the newly revealed dynamic controls after setting the scroll bar) before re-planning based on the actual UI state. DMI disallows mixing *visit* interface with other interfaces in the same turn to enforce this. An exception is *access-and-input-text*. We specifically include this command in the *visit* interface, which overlaps with state declaration, because the outcome of an “edit” action is limited and highly predictable, and therefore is unnecessary to observe the execution result. This design improves efficiency by allowing the execution of edit commands with other control-access commands in a single turn, without a forced stop to observe. Third, *visit* supports shortcut commands for essential keyboard functions (e.g., pressing ENTER to commit text in an edit field). We instruct the LLM to use this judiciously.

Handling unstable UI interaction: We design several techniques to enhance the robustness of the executor. First, we employ a fuzzy control matcher that combines control type, ancestor hierarchy, and name similarity when exact matching fails due to name variations or UIA’s lack of guaranteed unique identifiers. Second, we provide structured error feedback that explicitly describes control states and context to guide subsequent planning, e.g., when a target control is located but disabled. Finally, we use a failure retry mechanism for GUI controls that may load slowly, retrying when deterministically expected controls are absent after interactions. We exclude shortcut-key operations to prevent unintended side effects from repeated executions, e.g., pressing Enter twice.

Handling improper LLM instruction-following: Despite explicit instructions to output only target control identifiers, LLMs sometimes include navigational nodes. We address this by trusting the LLM’s intended destination while ignoring its navigation process. The key insight is that functional nodes are topology leaves, while navigational nodes are non-leaves. We filter out non-leaf nodes, allowing the executor to retain only commands targeting functional nodes

Table 2. *state* declaration and *observation* declaration interfaces. UIA defines 34 control patterns in total. These interfaces are extensible. For example, *set_texts* builds on *TextPattern*, *set_toggle_state* builds on *TogglePattern*, and *set_expanded*/*set_collapsed* builds on *ExpandCollapsePattern*.

Interface	Control Pattern	Description
<i>set_scrollbar_pos</i>	Scroll	Set scrollbar position to x%
<i>select_lines</i>	Text	Select one (or contiguous) line(s)
<i>select_paragraphs</i>	Text	Select one paragraph or a contiguous paragraph range
<i>select_controls</i>	Select	Single or multi-select controls
<i>get_texts</i>	Text&Value	Retrieve a control’s text

and handle navigation independently. This provides fault tolerance regardless of whether the LLM outputs navigational steps. Shortcut-key commands following filtered commands are also removed to maintain consistency.

3.5 State and Observation Declaration: Interaction-related Interfaces

Triggering a control’s functionality requires interacting with it. Beyond basic interactions (e.g., clicks), controls may require complex interactions involving coordinated keyboard and mouse actions (e.g., multi-select controls). Under the UIA framework, a control describes its available functionality through a finite set of control patterns (e.g., *TextPattern*, *ScrollPattern*, *SelectionPattern*). We leverage these control patterns and corresponding UIA interfaces to encapsulate control interactions, thereby providing a new layer of abstraction. In this way, we realize *state* declaration and *observation* declaration (see § 3.1).

state and *observation* declarations reduce dependency on precise visual perception and simplify complex interaction. For example, an Excel cell’s full content may not be fully visible and would require several clicks to reveal. DMI ships with a set of pre-wrapped, high-value, and complex interactions, and can be customized to add additional UIA control patterns accordingly (see Table 2).

Supporting precise perception by default: When DMI interacts with applications, controls may expose dynamic data that cannot be modeled offline. We adopt a “passive + active” design for *get_texts()*. Specifically, before each LLM call, *get_texts()* is invoked in passive mode on all *DataItem* controls, and a truncated, structured result is forwarded into the prompt. The passive mode reduces fine-grained visual parsing and saves round-trip, and items with empty values are coalesced for brevity. In cases where truncated results are insufficient, LLMs can call *get_texts()* in active mode to retrieve the full content. The operation is implemented on UIA *TextPattern* and *ValuePattern* and generalizes to non-*DataItem* controls.

Separating control access and complex interactions:

Control identifiers are used by interaction-related interfaces. However, to prevent mixing with visit interface within the same turn and preserve accuracy (see 3.4), the use of static IDs from the navigation topology is explicitly prohibited by interaction-related interfaces. Instead, those interfaces can only operate on controls specified by their label from the current screen’s accessibility tree.

4 Implementation

DMI comprises over 18K lines of Python code and leverages the pywinauto library [31] to exercise UIA ¹.

4.1 UNG Construction

This section describes the methodology for constructing the UNG, which is formally defined in § 3.2.

Control identifier synthesis: We require a control identifier to label each UI control as a node in the UNG. Since UIA lacks guaranteed globally unique identifiers, we adopt an XPath-like control identifier:

primary_id|control_type|ancestor_path

primary_id uses the UIA automation_id (if empty, falling back to the control name, or [Unnamed]); control_type specifies the UIA-defined type (e.g., TabItem); and ancestor_path provides a slash-delimited sequence of UI tree ancestors. We avoid index-based addressing since dynamic menus can shift indices unpredictably.

GUI ripping: The UNG models transition relationships between UI controls. The UNG is built via differential capture. Exploration proceeds with depth-first search (DFS). First, obtain the accessibility tree of the target application, from which a list of UI controls is captured. Then activate a candidate control (i.e., click) from the list and capture again. Newly revealed controls define navigation edges. New top-level or modal windows are detected via process_id and window listeners. The exploration is fully automated.

Access blacklist: The DFS-based ripping requires the application to return to the prior state before exploring new branches. However, certain controls (i) trigger external transitions (e.g., an “Account” button opening a Web browser) or (ii) bring the application into states that cannot be exited using standard commands like Esc or Close. While state consistency can be force-restored by restarting the application after every interaction [22], this incurs a prohibitive time cost. To bypass this bottleneck, we adopt a **semi-automated** approach by implementing a manual blacklist for these controls. The configuration leverages prior application knowledge and can be verified by monitoring the modeling process. This represents a deliberate trade-off: necessary human intervention in exchange for a massive gain in exploration efficiency. Maintaining the blacklist constitutes most of the

manual effort, but it can be automated with additional modeling time.

Context-aware exploration: Some controls are implicitly state-dependent and are only visible under specific conditions (e.g., PowerPoint’s “Picture Format” tab appears only when an image is selected). Accordingly, we implement a context manager. We manually instantiate representative objects (e.g., inserting an image or a text box into the slides) along with their associated context types (e.g., image, text box). The explorer traverses each context independently and merges the results into a unified topology. Although this manual setup represents a small fraction of the overall manual effort, automating it remains challenging. Omitting it would compromise the completeness of the UNG.

Root node initialization: A virtual root is introduced, and controls on the initial screen are attached as its children. If multiple TabItem controls exist and one is active by default, we associate otherwise unscoped controls on the initial screen with that active tab control to ensure they are indexable. The process is fully automated.

4.2 Descriptions of Controls and Navigation

Output schema: Each UI navigation tree and subtree is serialized as compact structured text:

name(type)(description)_id[children]

Parentheses mark optional fields; square brackets encode hierarchical nesting. The name, type, and description are drawn solely from UIA properties provided by the application. The id is a unique, consecutive integer used for concise references (replacing verbose control IDs). The children are a comma-separated list of child control information.

Truncating descriptions: Control descriptions are selectively attached and truncated (by characters or tokens). The full_description property is always included for control with key type (e.g., Menu, TabItem, ComboBox, Group, Button) when available. If multiple controls share a name and the group includes at least one key type, descriptions are applied to all. The full_description property is preferred; otherwise, a location-based identifier is used (§ 4.1). The non-leaf (navigational) nodes are fewer but pivotal; by default, full descriptions are included for these nodes when available.

4.3 The visit Interface

visit receives a JSON array of commands from the LLM, and translates each into concrete GUI or keyboard actions.

Path resolution: The executor first discards commands targeting non-leaf (navigational) nodes and any following shortcut_key commands, thereby retaining only the intended functional (leaf) targets. Each retained command is then resolved to a unique root-to-target navigation path.

¹DMI will be released at <https://github.com/dmi-interface/DMI>

Path navigation: The executor traverses this path from the current UI state. On each iteration, it fetches the topmost valid window and all descendant controls, and matches the parsed path from the end backward against the visible hierarchy. If no control from the remaining path exists in the current window, it closes that window. The “closing” follows a priority of OK > Close > Cancel, favoring the saving of modifications. Once a match is established, it proceeds forward along the path. Navigation robustness is enhanced by fuzzy matching and failure retries.

Control Interaction: After navigation, the executor performs the interaction based on the control access command, such as a click or a click followed by text input.

4.4 The interaction-related Interfaces

We adopt a conservative execution strategy for these interfaces. If any controls do not support the required pattern, the executor returns an error and does not partially execute. The executor returns a structured status (e.g., scrollbar positions).

5 Evaluation

Our key takeaways from the evaluation are:

- Compared to the GUI-only baseline, DMI increases success rates by **1.67×** and reduces steps by **43.5%** over several models and tasks.
- DMI enables global planning and allows LLMs to complete user intent using a **single** call (in >61% of cases).
- With DMI (GUI+DMI), most failures are **policy-level** (81%, semantic planning); only 19% are mechanism-level (navigation/interaction).

5.1 Setup

Case studies: We evaluate Microsoft Word, Excel, and PowerPoint as representative, feature-rich productivity applications. These applications collectively cover a range of scenarios, from text and tabular editing to graphics, and expose over 5,000 controls each. Their complex UI features nested dialogs, child windows, and dynamic panes, with a navigation depth exceeding 10. This results in a complex navigation topology with cycles and numerous merge nodes. The suite broadly covers UIA control types and patterns, and mixes structured controls with substantial unstructured content (e.g., arbitrary document text, free-form shapes on slides).

Benchmark: We draw tasks from the widely used OSWorld benchmark [44]; specifically, the OSWorld-W (Windows) portion comprising 27 single-app scenarios for PowerPoint, Excel, and Word. We evaluate on the full set of single-app scenarios. We exclude the multi-app subset because it exercises the operating system’s controls. Although modeling the OS controls is feasible, it would have required a significant amount of time for modeling.

Baseline: We adopt Microsoft’s Windows agent framework UFO-2 [51] as our baseline; it natively integrates UIA. For a fair comparison, we use the combination UFO2-base + action sequence (denoted UFO2-as). Here, UFO2-base is GUI-only. UFO-2 also offers Office-specific COM APIs, but those are not general and are therefore excluded. The action sequence mechanism reduces round trips by allowing the LLM to emit multiple actions in a single turn, provided all referenced controls are currently visible in the app UI. In the baseline (and all our settings), we register a UIA event handler to trigger applications to expose full control trees (avoiding lazy loading artifacts). We also change control labeling: before calling the LLM, the baseline labels accessible-tree controls and passes the labels in the prompt; to distinguish these labels from our numeric IDs in the navigation topology, labels are alphabetic (e.g., “A”, “HF”).

Our approach: On top of UFO2-as, we introduce our declarative DMI to evaluate its effectiveness. Prompts instruct the LLM to prefer DMI. When DMI cannot complete a step, the LLM may fall back to the baseline’s imperative GUI primitives (e.g., click, drag_on_coordinates, keyboard_input).

Methodology: Each task is capped at 30 steps to prevent excessive retries and is run three times, then averaged. Office experiments use Microsoft 365 builds. The LLMs are OpenAI GPT-5 and GPT-5-mini. The API variants are reasoning models with a configurable reasoning effort: minimal, low, medium (default), high. We compare GPT-5 at medium vs minimal to emulate “reasoning” vs “non-reasoning” modes. No fine-tuning is applied.

5.2 Offline Phase: UI Navigation Modeling Cost

Modeled graphs: We build the UNG with the prototype modeler and then transform it into a path-unambiguous forest. In the raw modeled graphs, Excel/PowerPoint/Word each exceeds 4K controls. To control context cost, we extract a core topology from the forest: Excel $\approx$ 2K, Word $\approx$ 1K, and PowerPoint $\approx$ 1K controls. Word and Excel take a forest; PowerPoint’s core topology is a single tree. No nested references exist among shared subtrees.

Cost: Automated modeling per application takes < 3 hours. Some manual setup is required (see § 4.1). We assume the operator has reasonable app proficiency (e.g., can identify controls that jump to external applications or drastically reshape the UI, either a priori or by observing the modeling execution). Average manual effort is around 1.5 person-days per application. The resulting model is version-specific but reusable across machines for the same application build.

5.3 Online Phase: End-to-end Performance

Terminology: Reasoning denotes the configured reasoning effect; SR is the average success rate. Step denotes the average number of LLM calls (i.e., round trips). Time is the

Table 3. Results across interfaces and models.

Interface	Knowledge	Model	Reasoning	SR	Steps	Time(s)
GUI-only	/	GPT-5	Medium	44.4%	8.16	392
GUI-only	Nav.forest	GPT-5	Medium	42.0%	8.41	353
GUI+DMI	Nav.forest	GPT-5	Medium	74.1%	4.61	239
GUI-only	/	GPT-5	Minimal	23.5%	8.42	251
GUI+DMI	Nav.forest	GPT-5	Minimal	40.7%	5.52	140
GUI-only	/	5-mini	Medium	17.3%	7.14	171
GUI-only	Nav.forest	5-mini	Medium	23.5%	6.32	150
GUI+DMI	Nav.forest	5-mini	Medium	43.2%	4.43	167

average completion time. All metrics are calculated on successful cases only (Table 3). We also report normalized steps (Figure 5b), which are calculated based on the intersection of tasks successfully solved by all compared methods (GUI Baseline, GUI+DMI, and Ablation).

Settings: We compare three settings: (1) GPT-5, reasoning mode (core setting); (2) GPT-5, minimal reasoning; and (3) GPT-5-mini, reasoning mode.

Overall improvement: In the core setting, DMI yields substantial improvements over the baseline: raising success from 44.4% to 74.1% (1.67 $\times$), cutting steps from 8.16 to 4.61 (−43.5%), and reducing completion time by 39%. Under minimal reasoning, success improves from 23.5% to 40.7%, steps drop from 8.42 to 5.52, and time declines from 251s to 140s. With GPT-5-mini, DMI yields an absolute +25.9% success gain (2.5 $\times$ over GUI-only) and 38% fewer steps; average time is comparable (171s baseline vs 167s DMI) because we report only successful runs, and GUI-only primarily succeeds on shorter/easier cases. Table 3 shows the overall results evaluated on the full set.

When normalized to the **intersection** of tasks solved by all methods (GUI-only, GUI+DMI, and Ablation), DMI demonstrates even greater efficiency gains. On this common subset, DMI reduces the normalized steps from 7.94 (GUI-only) and 8.58 (Ablation) to 4.60 (GUI+DMI) under the GPT-5 (medium reasoning) setting. This further confirms that DMI significantly reduces the interaction overhead (see Figure 5b). It is also important to note that tasks solved without DMI (GUI-only) **remain solvable** with DMI (GUI+DMI) in our evaluation.

One-shot task completion: In the core setting, with DMI, over 61% of successful trials are completed in 4 steps. The architecture of the UFO-2 framework features a multi-agent design where a HostAgent orchestrates the overall workflow while specialized AppAgents manage tasks for individual applications. UFO-2’s workflow incurs a fixed 3-step overhead. These 4 steps correspond to: (1) HostAgent decomposes the user task by application and opens/activates the target app. (2) APPAgent executes the delegated subtask. (3) APPAgent

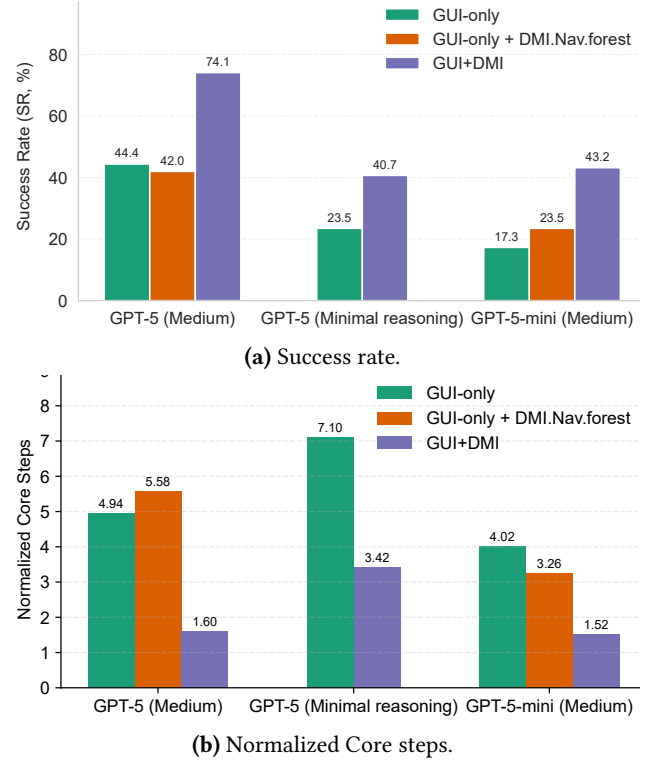


Figure 5. Visualization of performance evaluation. Core steps refer to the steps without the fixed 3-step agent framework overhead (see § 5.3 One-shot task completion). Normalization refers to the intersection of tasks that all methods solve.

verifies the result and decides on handoff. (4) HostAgent verifies overall task completion. Thus, for single-app tasks, DMI enables the LLM to complete the core user intent in a single LLM call. Although the baseline supports action sequences, it is constrained to currently visible controls and cannot plan over controls that are not yet exposed. Declarative DMI permits global planning, which cuts the number of LLM calls substantially, as depicted in Figure 5b.

5.4 Overhead

Token cost: Compared with the baseline, DMI introduces additional context tokens. Over 80% of this overhead comes from the navigation forest; the remainder stems from the DMI usage prompt and the truncated DataItem payloads returned by `get_texts()`. Empirically, each control contributes 15 tokens on average (measured under the `o200k_base` encoding). The core topologies add approximately 30K (Excel), 15K (Word), and 15K (PowerPoint) tokens. While this increases per-call context, modern LLMs provide ample windows (e.g., GPT-5 400K, o3 200K, Claude Sonnet 4 200K). More importantly, DMI reduces interaction rounds substantially, resulting in total token usage per task being lower than the baseline in the core scenario.

Per-step latency: With the small model, per-call latency is higher, but overall task completion time drops because DMI requires fewer steps. Per-call latency with DMI is similar to the baseline for large models in both reasoning and non-reasoning modes.

5.5 Ablation Study

To determine whether DMI’s performance gains stem from its declarative interface or simply from providing a static knowledge base, we conducted an ablation study on both large and small models, as shown in Table 3 and Figure 5.

For the baseline UFO2-as, we provided the DMI navigation forest in the prompt while disabling the declarative interface. This configuration yields no significant change in performance (SR 42% vs. 44% baseline; Steps 8.41 vs. 8.16, Normalized Steps 8.58 vs. 7.94), indicating that the declarative interface, not the static knowledge, is the primary source of DMI’s effectiveness. In contrast, the less capable GPT-5-mini model shows a modest improvement when given the forest alone (SR 23.5% vs. 17.3% baseline; Steps 6.32 vs. 7.14; Normalized Steps 6.26 vs. 7.02). This suggests that supplementary topology knowledge can be helpful for models with less general-purpose knowledge. Crucially, however, the full DMI setting produced much larger gains (SR 43.2%; Steps 4.43; Normalized Steps 4.52), confirming that the interface design is the dominant performance driver. The average time is slightly lower as the baseline succeeds on short/easy cases.

5.6 Failure Analysis

We analyze failures in the core setting and compare distributions between DMI and a GUI-only baseline (see Figure 6). The analysis combines execution results with a chain-of-thought summary returned by the LLM.

GUI+DMI (policy-centric failures): The dominant causes are: ambiguous task descriptions (42.9%); misinterpretation of control semantics (28.6%; e.g., Word’s “subscript” control in “Find and Replace” dialog applies to the whole Edit field rather than the selected range; Excel’s rule of “conditional formatting” applies to all cells in the selected region, including blank cells); weak visual-semantic understanding (14.3%); misunderstanding of subtle task semantics (9.5%); and topology/modeling inaccuracies (4.8%, see § 6 (In)accurate navigation topology).

Under the proposed method, errors concentrate at the policy level, specifically in task-semantic misunderstandings (52.4%) and misperception of control functionality (28.6%). This shift indicates that the declarative interface largely removes the mechanism-induced failures (navigation and interaction), leaving errors primarily in semantic understanding and planning, validating the intended decoupling.

GUI-only baseline (mechanism-dominated failures): The baseline shows many mechanism failures: control localization and navigation errors (14/45), and composite-interaction

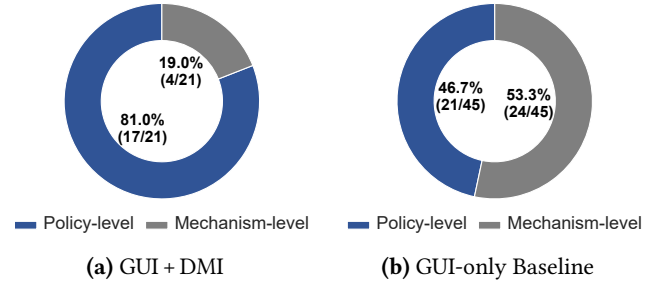


Figure 6. Failure-cause distribution (Policy/Mechanism) for DMI (GUI+DMI) and GUI-only baseline.

errors (7/45). It also shows additional task-semantic errors and misperception of control functionality (6/45). A further 18 failures overlap with the policy-centric categories above. Because the LLM must split attention between policy and mechanism, more semantic mistakes appear. We observed similar errors in the ablation experiment, reinforcing that coupling policy with mechanism leads to avoidable mistakes.

Summary: The failure analysis pinpoints why DMI works: it cuts out mechanism-related errors (navigation and composite interaction) and re-centers policy, where LLMs are comparatively strong. Compared to GUI, DMI aligns better with LLM capabilities, substantially reducing reliance on precise vision and high-frequency interactions.

5.7 Lessons Learned

We discuss lessons learned when developing suitable declarative interfaces for LLMs. The following elements are necessary for DMI to achieve high success rates and efficiency.

Global unique identifier: A critical lesson is the need for applications to expose globally unique identifiers for controls. Although UIA recommends AutomationId, its global uniqueness is not enforced. As a result, current control identification methods like XPath are not reliable.

Rich control descriptions: Comprehensive and explicit descriptions in accessibility metadata are crucial for effective computer-use agents. For example, Excel’s Name Box should explicitly state that pressing Enter is required to commit input. Future work can augment the textual navigation topology with descriptions synthesized from documentation or curated by LLMs.

Explicit navigation-node access: Our visit interface filters non-leaf (navigation) nodes, ensuring the interface takes over all navigation. If a task truly requires accessing a navigation node, the LLM can fall back to GUI operations. One alternative is to add an enforced parameter to control-access commands, allowing the caller to bypass filtering.

6 Discussion and Limitations

Completeness of DMI primitives: The mechanism of GUI-use can be reduced to two parts: (1) making the target control visible on the screen, and (2) interacting with the control. DMI leverages the determinism of GUI navigation and abstracts control interactions into state representation to simplify complexity. It is important to note that DMI cannot simplify all complex interactions. For example, tasks like freely drawing a shape (e.g., canvas) or position-precise manipulation (e.g., figure adjustment) are inherently dependent on fine-grained, **non-standard** position-related operations. Such tasks are difficult to express in a stateful manner and cannot be easily simplified using a declarative approach.

Together, DMI and GUI form a fast-path/slow-path design. DMI is the fast path used by default; GUI is the slow-path fallback, invoked whenever (i) the needed interaction is outside DMI’s coverage (e.g., freehand drawing) or (ii) execution hits exceptions/unexpected UI states (e.g., error pop-ups), ensuring functional completeness and recovery.

Generalization to other OSes: Our current implementation uses the Windows UI Automation (UIA) framework. We find that similar accessibility-based frameworks exist across OSes. macOS provides NSAccessibility that supports traversing and invoking UI elements across applications. Its role-subrole hierarchies and associated attributes/actions map naturally to UIA control patterns. Ubuntu exposes application widgets via AT-SPI2 [5]. This mechanism also provides callable interfaces (e.g., Action, Value, Text, Selection) functionally equivalent to UIA control patterns. Android AccessibilityService and AccessibilityNodeInfo provide comparable control pattern functionality.

Generalization to new applications: Extending DMI to new applications necessitates constructing an application-specific UNG to enable the access declaration based on navigation topology. As detailed in § 4.1, this involves manual configuration to facilitate the topology modeling.

Unsupported GUI applications: DMI is built atop Windows UI Automation (UIA), which also benefits many modern applications such as Google Chrome and Visual Studio Code. However, UIA does not cover all Windows applications. Legacy applications from the Windows XP era often lack UIA support. Real-time applications like video games and specialized graphics software often bypass standard UI widgets and may not expose accessible UIA trees, falling outside our methodology’s scope.

(In)accurate navigation topology: Task success depends critically on accurate navigation topology modeling. Our prototype infers edges by comparing accessible trees before and after a click action, but cannot capture all dependencies. For example, in Word’s “Find and Replace” dialog, entering special text (e.g., +1, +2, +3) into the “Rich Edit” control

dynamically renames the “Next” button to “Go To”. Depth-first search exploration may miss such conditional changes, and without stable application identifiers, name changes break element matching. When topology is incomplete, LLMs can fall back to GUI-based imperative primitives, but may still fail due to poor visual localization of LLMs.

Handling dynamic controls: Applications might be driven by a dynamic UI that resists comprehensive pre-modeling. A canonical example is the Web browser. The main challenge of adapting DMI to Web browsers is the high volatility of Web controls—DOM structures and element attributes frequently change due to dynamic content rendering and continuous updates. This complicates navigation abstraction (access declaration) compared to relatively stable desktop applications. Handling dynamic controls is a challenging and promising direction for future work. Nevertheless, DMI’s state and observation declarations target UI controls interaction (e.g., set a scrollbar position, checkbox state, text in edit_box), and thus can smoothly adapt to browsers.

7 Related Work

GUI-based large action models: UI-TARS [32], OS-ATLAS [43], Aguis [45], and UGround [11] train specialized large action models to navigate human-oriented GUI better. In contrast, DMI redesigns the interface to be LLM-friendly, eliminating the need for model training or fine-tuning.

Skill-based approaches: AppAgentX [17] encapsulates UI sequences as skills. However, its reliance on visual element matching introduces a fundamental fragility. Similarly, AXIS [20] translates UI traces into application API calls, but its robustness is contingent upon the availability and stability of those APIs. In contrast, our accessibility-based interfaces offer a more robust approach. It operates without the need for app-specific APIs. Furthermore, our approach enables the composition of complex skills using declarative primitives, circumventing the need to rely on low-level UI sequences.

GUI ripping: Several studies in GUI testing model the application’s structure to generate test cases. GUITAR [22] abstracts GUIs into event-flow graphs and synthesizes test cases from the model. Exploration strategies range from DFS to randomized search; platform-specific analogues include Android’s monkey tool [35]. Broadly, some approaches [39, 41] model abstract state transitions, while others model reachability among GUI controls [24, 42]. The key difference is how we treat path ambiguity at scale. Early Win32 GUITAR [22] expands multi-source graphs into multiple trees by cloning subgraphs to ensure unique paths; whereas modern, large UIs trigger exponential node growth. We instead introduce a cost-aware path disambiguation that yields a forest (namely, a main tree and shared subtrees) with controlled size.

Graph-based methods: AutoDroid-v1/v2 [41, 42] constructs UI/Element transition graphs and fine-tunes an on-device language model (LM) with sampled training data, which enables the LM to generate action sequences even when controls are not currently visible. However, this approach remains fundamentally imperative; the LM must plan both the high-level policy and the low-level mechanism, requiring it to enumerate fine-grained, step-by-step navigation sequences (e.g., `tap(a) -> tap(b) -> tap(c)`). When the LM’s predicted next control is not found, the reachable path closest to that control is picked. However, this approach suffers from path ambiguity. For example, a single control labeled “Color” could correspond to different semantic functions (e.g., font color, outline color, or underline color). The greedy method does not reliably resolve such ambiguity. This poses a significant risk of incorrect functionality. In contrast, DMI provides unambiguous control access.

Novel system interfaces: There is a long history of efforts to design new system interfaces. Commutative interfaces [9] enhance OS multi-core scalability. SLO-aware interface [12] achieves millisecond tail tolerance. [16, 21] proposes performance interfaces for systems. Moreover, [36] examines modern Linux APIs’ usage and compatibility, providing insights for new system interface design.

Other related work: SWE-agent [46] defines Agent-Computer Interface (ACI) to enhance software engineering. Agent S [3] proposes another ACI utilizing Accessibility APIs to label on-screen controls (similar to our baseline [51]). Agent S2 [4] introduces the Mixture-of-Grounding that equips the LLM with specialized tools to enhance CUAs. SeeClick [7] uses large vision-language models to locate controls.

8 Conclusion

OSes have long evolved interfaces to serve different users—from command-line interface (CLI) for experts to GUI for general users. LLM-powered agents are a fundamentally new class of OS users with distinct characteristics (large context-memory, reasoning) and constraints (weak visual capability, latency, token costs, imperfect instruction-following) that existing OS interfaces do not consider or address. As a result, LLMs underperform with GUI.

DMI addresses this gap by applying core systems principles (declarative over imperative, separating policy from mechanism, fast-path/slow-path) to build LLM-friendly OS interfaces.

Acknowledgments

We sincerely thank the anonymous reviewers of EuroSys 2026 for their constructive comments, and our shepherd Jayson Boubin, for reviewing our camera-ready version. We also thank Zhao Zhang and Zehao Song for proofreading drafts. This work was supported in part by the National

Natural Science Foundation of China (No. 62502510). Mingyu Li (limingyu@ios.ac.cn) and Haibo Chen (haibochen@ios.ac.cn) are the corresponding authors.

References

- [1] Reyna Abhyankar, Qi Qi, and Yiyang Zhang. 2025. OSWorld-Gold: Benchmarking the Efficiency of Computer-Use Agents. In *ICML 2025 Workshop on Computer Use Agents*. <https://openreview.net/forum?id=sV3n6mYy7J>
- [2] OS Accessibility. 2023. <https://www.w3.org/TR/wai-aria/#dfn-accessibility-api>
- [3] Saaket Agashe, Jiuzhou Han, Shuyu Gan, Jiachen Yang, Ang Li, and Xin Eric Wang. 2025. Agent S: An Open Agentic Framework that Uses Computers Like a Human. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net. <https://openreview.net/forum?id=llVRgt4nLv>
- [4] Saaket Agashe, Kyle Wong, Vincent Tu, Jiachen Yang, Ang Li, and Xin Eric Wang. 2025. Agent S2: A Compositional Generalist-Specialist Framework for Computer Use Agents. In *Second Conference on Language Modeling*. <https://openreview.net/forum?id=Zg5is4Gj3R>
- [5] Linux AT-SPI2. 2022. <https://gnome.pages.gitlab.gnome.org/at-spi2-core/devel-docs/architecture.html>
- [6] Windows UI Automation. 2025. <https://learn.microsoft.com/en-us/windows/win32/winauto/entry-uaui-win32>
- [7] Kanzhi Cheng, Qishi Sun, Yougang Chu, Fangzhi Xu, Yantao Li, Jianbing Zhang, and Zhiyong Wu. 2024. SeeClick: Harnessing GUI Grounding for Advanced Visual GUI Agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2024, Bangkok, Thailand, August 11-16, 2024*. Association for Computational Linguistics, 9313–9332. doi:10.18653/V1/2024.ACL-LONG.505
- [8] Krishna Teja Chitty-Venkata, Siddhisanket Raskar, Bharat Kale, Farah Ferdaus, Aditya Tanikanti, Ken Raffanetti, Valerie Taylor, Murali Emani, and Venkatram Vishwanath. 2024. Llm-inference-bench: Inference benchmarking of large language models on ai accelerators. In *SC24-W: Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 1362–1379.
- [9] Austin T. Clements, M. Frans Kaashoek, Nickolai Zeldovich, Robert T. Morris, and Eddie Kohler. 2013. The scalable commutativity rule: designing scalable software for multicore processors. In *ACM SIGOPS 24th Symposium on Operating Systems Principles, SOSP '13, Farmington, PA, USA, November 3-6, 2013*. ACM, 1–17. doi:10.1145/2517349.2522712
- [10] Yao Fu, Leyang Xue, Yeqi Huang, Andrei-Octavian Brabete, Dmitrii Ustiugov, Yuvraj Patel, and Luo Mai. 2024. ServerlessLLM: Low-Latency Serverless Inference for Large Language Models. In *18th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2024, Santa Clara, CA, USA, July 10-12, 2024*. USENIX Association, 135–153. <https://www.usenix.org/conference/osdi24/presentation/fu>
- [11] Boyu Gou, Ruohan Wang, Boyuan Zheng, Yanan Xie, Cheng Chang, Yiheng Shu, Huan Sun, and Yu Su. 2025. Navigating the Digital World as Humans Do: Universal Visual Grounding for GUI Agents. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net. <https://openreview.net/forum?id=kxnoqaisCT>
- [12] Mingzhe Hao, Huaicheng Li, Michael Hao Tong, Chrisma Pakha, Riza O. Suminto, Cesar A. Stuardo, Andrew A. Chien, and Haryadi S. Gunawi. 2017. MittOS: Supporting Millisecond Tail Tolerance with Fast Rejecting SLO-Aware OS Interface. In *Proceedings of the 26th Symposium on Operating Systems Principles, Shanghai, China, October 28-31, 2017*. ACM, 168–183. doi:10.1145/3132747.3132774
- [13] Hongliang He, Wenlin Yao, Kaixin Ma, Wenhao Yu, Yong Dai, Hongming Zhang, Zhenzhong Lan, and Dong Yu. 2024. WebVoyager: Building an End-to-End Web Agent with Large Multimodal Models. In

- Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2024, Bangkok, Thailand, August 11–16, 2024.* Association for Computational Linguistics, 6864–6890. doi:10.18653/V1/2024.ACL-LONG.371
- [14] Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Yu, Xinying Song, and Denny Zhou. 2024. Large Language Models Cannot Self-Correct Reasoning Yet. In *International Conference on Representation Learning*, Vol. 2024. 32808–32824. https://proceedings.iclr.cc/paper_files/paper/2024/file/8b4add8b0aa8749d80a34ca5d941c355-Paper-Conference.pdf
 - [15] Zhiyuan Huang, Ziming Cheng, Junting Pan, Zhaohui Hou, and Mingjie Zhan. 2025. SpiritSight Agent: Advanced GUI Agent with One Look. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2025, Nashville, TN, USA, June 11–15, 2025*. Computer Vision Foundation / IEEE, 29490–29500. doi:10.1109/CVPR52734.2025.02746
 - [16] Rishabh R. Iyer, Katerina J. Argyraki, and George Candea. 2022. Performance Interfaces for Network Functions. In *19th USENIX Symposium on Networked Systems Design and Implementation, NSDI 2022, Renton, WA, USA, April 4–6, 2022*. USENIX Association, 567–584. <https://www.usenix.org/conference/nsdi22/presentation/iyer>
 - [17] Wenjia Jiang, Yangyang Zhuang, Chenxi Song, Xu Yang, and Chi Zhang. 2025. AppAgentX: Evolving GUI Agents as Proficient Smartphone Users. *CoRR* abs/2503.02268 (2025). arXiv:2503.02268 doi:10.48550/ARXIV.2503.02268
 - [18] Hanyu Lai, Xiao Liu, Iat Long Iong, Shuntian Yao, Yuxuan Chen, Pengbo Shen, Hao Yu, Hanchen Zhang, Xiaohan Zhang, Yuxiao Dong, and Jie Tang. 2024. AutoWebGLM: A Large Language Model-based Web Navigating Agent. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, KDD 2024, Barcelona, Spain, August 25–29, 2024*. ACM, 5295–5306. doi:10.1145/3637528.3671620
 - [19] Hanyu Lai, Xiao Liu, Yanxiao Zhao, Han Xu, Hanchen Zhang, Bohao Jing, Yanyu Ren, Shuntian Yao, Yuxiao Dong, and Jie Tang. 2026. ComputerRL: Scaling End-to-End Online Reinforcement Learning for Computer Use Agents. In *The Fourteenth International Conference on Learning Representations*. <https://openreview.net/forum?id=oEVFn0w4B>
 - [20] Junting Lu, Zhiyang Zhang, Fangkai Wang, Jue Zhang, Lu Wang, Chao Du, Qingwei Lin, Saravan Rajmohan, Dongmei Zhang, and Qi Zhang. 2025. AXIS: Efficient Human-Agent-Computer Interaction with API-First LLM-Based Agents. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2025, Vienna, Austria, July 27 - August 1, 2025*. Association for Computational Linguistics, 7711–7743. <https://aclanthology.org/2025.acl-long.381/>
 - [21] Jiacheng Ma, Rishabh R. Iyer, Sahand Kashani, Mahyar Emami, Thomas Bourgeat, and George Candea. 2024. Performance Interfaces for Hardware Accelerators. In *18th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2024, Santa Clara, CA, USA, July 10–12, 2024*. USENIX Association, 855–874. <https://www.usenix.org/conference/osdi24/presentation/ma-jiacheng>
 - [22] Atif M. Memon, Ishan Banerjee, and Adithya Nagarajan. 2003. GUI Ripping: Reverse Engineering of Graphical User Interfaces for Testing. In *10th Working Conference on Reverse Engineering, WCRE 2003, Victoria, Canada, November 13–16, 2003*. IEEE Computer Society, 260–269. doi:10.1109/WCRE.2003.1287256
 - [23] Liang Mi, Weijun Wang, Wenming Tu, Qingfeng He, Rui Kong, Xinyu Fang, Yazhu Dong, Yikang Zhang, Yuan Chun Li, Meng Li, Haipeng Dai, Guihai Chen, and Yunxin Liu. 2025. Empower Vision Applications with LoRA LMM. In *Proceedings of the Twentieth European Conference on Computer Systems (Rotterdam, Netherlands) (EuroSys '25)*. Association for Computing Machinery, New York, NY, USA, 261–277. doi:10.1145/3689031.3717472
 - [24] Inês Coimbra Morgado, Ana CR Paiva, and João Pascoal Faria. 2012. Dynamic reverse engineering of graphical user interfaces. *International Journal On Advances in Software* 5, 3 (2012), 224–236.
 - [25] Runliang Niu, Jindong Li, Shiqi Wang, Yali Fu, Xiyu Hu, Xueyuan Leng, He Kong, Yi Chang, and Qi Wang. 2024. ScreenAgent: A Vision Language Model-driven Computer Control Agent. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence (IJCAI-2024)*. International Joint Conferences on Artificial Intelligence Organization, 6433–6441. doi:10.24963/ijcai.2024/711
 - [26] OpenAI Operator. 2025. <https://openai.com/index/introducing-operator/>
 - [27] OpenAI Structured Outputs. 2025. <https://platform.openai.com/docs/guides/structured-outputs>
 - [28] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. 2024. Gorilla: Large Language Model Connected with Massive APIs. In *Advances in Neural Information Processing Systems*, A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang (Eds.), Vol. 37. Curran Associates, Inc., 126544–126565. doi:10.52202/079017-4020
 - [29] UIA Control Pattern. 2025. <https://learn.microsoft.com/en-us/windows/win32/wiauto/uiauto-controlpatternsoverview>
 - [30] Microsoft GUI Interface Design Principle. 2025. <https://learn.microsoft.com/en-us/power-platform/well-architected/experience-optimization/interaction-design>
 - [31] pywinauto. 2025. <https://github.com/pywinauto/pywinauto>
 - [32] Yujia Qin, Yining Ye, Junjie Fang, Haoming Wang, Shihao Liang, Shizuo Tian, Junda Zhang, Jiahao Li, Yunxin Li, Shijue Huang, Wan Jun Zhong, Kuanye Li, Jiale Yang, Yu Miao, Woyu Lin, Longxiang Liu, Xu Jiang, Qianli Ma, Jingyu Li, Xiaojun Xiao, Kai Cai, Chuang Li, Yaowei Zheng, Chaolin Jin, Chen Li, Xiao Zhou, Minchao Wang, Haoli Chen, Zhaojian Li, Haihua Yang, Haifeng Liu, Feng Lin, Tao Peng, Xin Liu, and Guang Shi. 2025. UI-TARS: Pioneering Automated GUI Interaction with Native Agents. arXiv:2501.12326 [cs.AI] <https://arxiv.org/abs/2501.12326>
 - [33] Zhuocheng Shen. 2024. LLM With Tools: A Survey. arXiv:2409.18807 [cs.AI] <https://arxiv.org/abs/2409.18807>
 - [34] Ben Shneiderman. 1981. Direct manipulation: A step beyond programming languages (abstract only). *SIGSOC Bull.* 13, 2–3 (May 1981), 143. doi:10.1145/1015579.810991
 - [35] Android Monkey Tool. 2023. <https://developer.android.com/studio/test/other-testing-tools/monkey?hl=en>
 - [36] Chia-Che Tsai, Bhushan Jain, Nafees Ahmed Abdul, and Donald E. Porter. 2016. A study of modern Linux API usage and compatibility: what to support when you're supporting. In *Proceedings of the Eleventh European Conference on Computer Systems, EuroSys 2016, London, United Kingdom, April 18–21, 2016*. ACM, 16:1–16:16. doi:10.1145/2901318.2901341
 - [37] Claude Computer Use. 2024. <https://docs.claude.com/en/docs/agents-and-tools/tool-use/computer-use-tool>
 - [38] Claude Tool Use. 2025. <https://www.anthropic.com/engineering/advanced-tool-use>
 - [39] Tanja EJ Vos, Peter M Kruse, Nelly Condori-Fernández, Sebastian Bauersfeld, and Joachim Wegener. 2015. Testar: Tool support for test automation at the user interface level. *International Journal of Information System Modeling and Design (IJISMD)* 6, 3 (2015), 46–83.
 - [40] Shuai Wang, Weiwen Liu, Jingxuan Chen, Weinan Gan, Xingshan Zeng, Shuai Yu, Xinlong Hao, Kun Shao, Yasheng Wang, and Ruiming Tang. 2024. GUI Agents with Foundation Models: A Comprehensive Survey. *CoRR* abs/2411.04890 (2024). arXiv:2411.04890 doi:10.48550/ARXIV.2411.04890
 - [41] Hao Wen, Yuanchun Li, Guohong Liu, Shanhui Zhao, Tao Yu, Toby Jia-Jun Li, Shiqi Jiang, Yunhao Liu, Yaqin Zhang, and Yunxin Liu. 2024. AutoDroid: LLM-powered Task Automation in Android. In *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking, ACM MobiCom 2024, Washington D.C., DC, USA, November 18–22, 2024*. ACM, 543–557. doi:10.1145/3636534.3649379
 - [42] Hao Wen, Shizuo Tian, Borislav Pavlov, Wenjie Du, Yixuan Li, Ge Chang, Shanhui Zhao, Jiacheng Liu, Yunxin Liu, Ya-Qin Zhang, and Yuanchun Li. 2025. AutoDroid-V2: Boosting SLM-based GUI Agents

- via Code Generation. In *Proceedings of the 23rd Annual International Conference on Mobile Systems, Applications and Services, MobiSys 2025, Hilton Anaheim, Anaheim, CA, USA, June 23-27, 2025*, Ardalan Amiri Sani, Chenren Xu, Przemyslaw Pawelczak, and Swarun Kumar (Eds.). ACM, 223–235. doi:10.1145/3711875.3729134
- [43] Zhiyong Wu, Zhenyu Wu, Fangzhi Xu, Yian Wang, Qiushi Sun, Chengyou Jia, Kanzhi Cheng, Zichen Ding, Liheng Chen, Paul Pu Liang, and Yu Qiao. 2025. OS-ATLAS: Foundation Action Model for Generalist GUI Agents. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net. <https://openreview.net/forum?id=n9PDaFNi8t>
- [44] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. 2024. OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*. http://papers.nips.cc/paper_files/paper/2024/hash/5d413e48f84dc61244b6be550f1cd8f5-Abstract-Datasets_and_Benchmarks_Track.html
- [45] Yiheng Xu, Zekun Wang, Junli Wang, Dunjie Lu, Tianbao Xie, Amrita Saha, Doyen Sahoo, Tao Yu, and Caiming Xiong. 2025. Aguis: Unified Pure Vision Agents for Autonomous GUI Interaction. In *Forty-second International Conference on Machine Learning, ICML 2025, Vancouver, BC, Canada, July 13-19, 2025 (Proceedings of Machine Learning Research, Vol. 267)*, Aarti Singh, Maryam Fazel, Daniel Hsu, Simon Lacoste-Julien, Felix Berkenkamp, Tegan Maharaj, Kiri Wagstaff, and Jerry Zhu (Eds.). PMLR / OpenReview.net. <https://proceedings.mlr.press/v267/xu25ae.html>
- [46] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*. http://papers.nips.cc/paper_files/paper/2024/hash/5a7c947568c1b1328ccc5230172e1e7c-Abstract-Conference.html
- [47] Yan Yang, Dongxu Li, Yutong Dai, Yuhao Yang, Ziyang Luo, Zirui Zhao, Zhiyuan Hu, JUNZHE HUANG, Amrita Saha, Zeyuan Chen, Ran Xu, Liyuan Pan, Caiming Xiong, and Junnan Li. 2026. GTA1: GUI Test-time Scaling Agent. In *The Fourteenth International Conference on Learning Representations*. <https://openreview.net/forum?id=3VIPmz7iAi>
- [48] Jiabo Ye, Xi Zhang, Haiyang Xu, Haowei Liu, Junyang Wang, Zhaoqing Zhu, Ziwei Zheng, Feiyu Gao, Junjie Cao, Zhengxi Lu, Jitong Liao, Qi Zheng, Fei Huang, Jingren Zhou, and Ming Yan. 2025. Mobile-Agent-v3: Fundamental Agents for GUI Automation. *CoRR* abs/2508.15144 (2025). arXiv:2508.15144 doi:10.48550/ARXIV.2508.15144
- [49] Chaoyun Zhang, Shilin He, Liqun Li, Si Qin, Yu Kang, Qingwei Lin, Saravan Rajmohan, and Dongmei Zhang. 2025. API Agents vs. GUI Agents: Divergence and Convergence. In *ICML 2025 Workshop on Computer Use Agents*. <https://openreview.net/forum?id=XuThXhPSQR>
- [50] Chaoyun Zhang, Shilin He, Jiaxu Qian, Bowen Li, Liqun Li, Si Qin, Yu Kang, Minghua Ma, Guyue Liu, Qingwei Lin, Saravan Rajmohan, Dongmei Zhang, and Qi Zhang. 2025. Large Language Model-Brained GUI Agents: A Survey. *Transactions on Machine Learning Research* (2025). <https://openreview.net/forum?id=xChvYjvXTp>
- [51] Chaoyun Zhang, He Huang, Chiming Ni, Jian Mu, Si Qin, Shilin He, Lu Wang, Fangkai Yang, Pu Zhao, Chao Du, Liqun Li, Yu Kang, Zhao Jiang, Suzhen Zheng, Rujia Wang, Jiaxu Qian, Minghua Ma, Jian-Guang Lou, Qingwei Lin, Saravan Rajmohan, and Dongmei Zhang. 2025. UFO2: The Desktop AgentOS. arXiv:2504.14603 [cs.AI] <https://arxiv.org/abs/2504.14603>
- [52] Chaoyun Zhang, Liqun Li, Shilin He, Xu Zhang, Bo Qiao, Si Qin, Minghua Ma, Yu Kang, Qingwei Lin, Saravan Rajmohan, Dongmei Zhang, and Qi Zhang. 2025. UFO: A UI-Focused Agent for Windows OS Interaction. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL 2025 - Volume 1: Long Papers, Albuquerque, New Mexico, USA, April 29 - May 4, 2025*. Association for Computational Linguistics, 597–622. doi:10.18653/V1/2025.NAACL-LONG.26